

Discriminative Project – Milestone 2 Report

Multi-Object Detection with YOLOv8: Identifying and Locating All 73 Object IDs

Sankalp Shinde · Study Group 8 · October 2026

Code: <https://github.com/imangoman/IE7615-Object-Recognition>

Summary. 4,000 multi-object images (2–9 objects each) were generated by concatenating randomly selected single-object photos from the final merged dataset, and a COCO-pretrained YOLOv8s model was trained to detect all 73 Object IDs. On 500 held-out test images, the model reached **mAP@0.5 = 0.821** and **mAP@0.5:0.95 = 0.754**. At a confidence threshold of 0.25, it found **81% of all objects with the correct Object ID and location**, and **97.7% of the objects it located were given the correct ID**. It also identified all 12 objects in an unseen 4×3 layout larger than anything seen in training.

1. Task and Approach

Given an image containing several of the class-collected objects, the model must report the **Object ID and location (bounding box) of every object** present. The final Object IDs from the shared sheet (OBJ001–OBJ073) are used throughout. The approach has three stages:

- **Generate a multi-object dataset** by concatenating randomly selected single-object images (Section 2).
- **Train YOLOv8** on it, using the Object IDs as class names (Section 3).
- **Evaluate** on held-out test images and demonstrate detection on new images (Sections 4 and 5).

2. Multi-Object Dataset Generation

2.1 Single-object source images

The single-object photos were cleaned exactly as in Milestone 1: all 7,343 images were validated, 28 exact duplicates were removed, and images were resized to 224×224, leaving 7,315 images. These were split 70/15/15 into training, validation and test sets, stratified by Object ID. To give each object a tight box inside its photo, OWLv2, an open-vocabulary detector, was prompted with the object's name (for example, “a photo of a Basketball”). If OWLv2 was not confident, the whole photo was used as the box; this fallback applied to 13.6% of photos.

2.2 Building the composite images

Each 640×640 training image is built as follows:

- **Layout:** a random grid of 1×2, 2×1, 1×3, 3×1, 2×2, 2×3, 3×2 or 3×3 cells, with larger grids chosen more often. Each cell is left empty with 15% probability, giving 2–9 objects per image in varied positions.
- **Object selection:** each cell receives a single-object photo from a **class-balanced pool**, which uses every photo once before repeating any, and never places the same Object ID twice in one image.
- **Augmentation:** each object is randomly scaled (55–100% of its cell), randomly positioned within the cell, flipped horizontally with 50% probability, and given ±20% brightness, contrast and saturation jitter. The background color is random.

- **Labels:** each object's box is its single-photo box mapped to where it was pasted, so box positions are known exactly by construction.

Train, validation and test composites are built only from photos in the matching Milestone 1 split. A check in the code confirms that no photo appears in more than one split, so the test set measures performance on photos the model has never seen.

Split	Images	Objects	Objects / image	Object IDs
Train	3,000	13,589	4.53 (2–9)	73
Validation	500	2,343	4.69 (2–9)	73
Test	500	2,309	4.62 (2–9)	73

Table 1. Generated multi-object dataset. Each Object ID has 172–213 training instances (median 185), so the classes are well balanced.

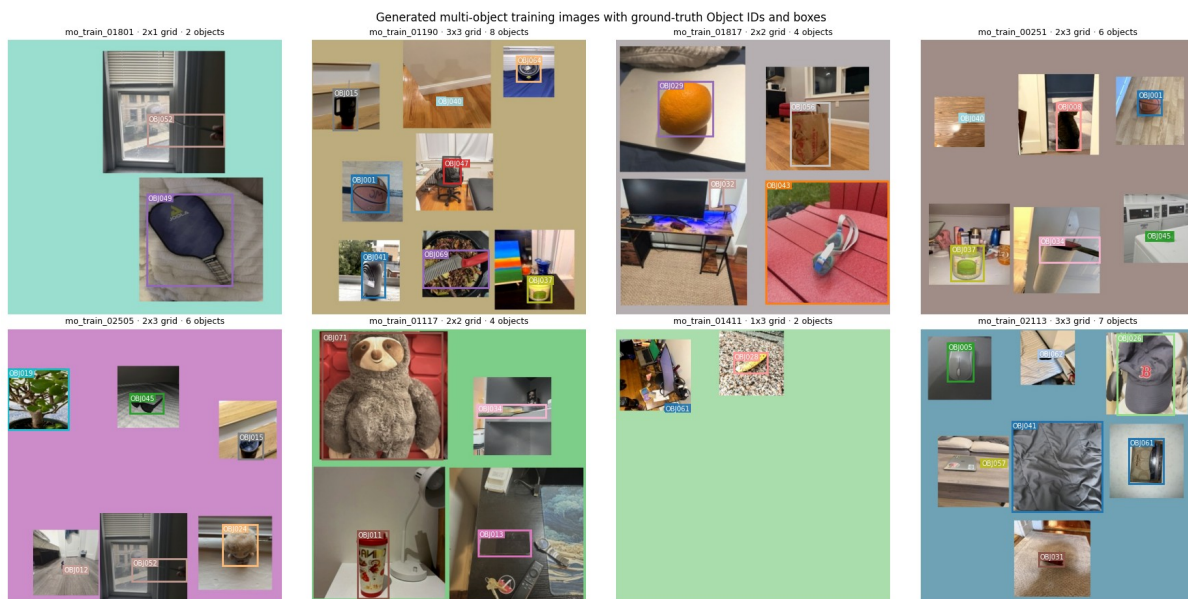


Figure 1. Generated training images with their ground-truth Object IDs and boxes.

3. Model Training

A **YOLOv8s** model (11.2 M parameters), pretrained on COCO, was fine-tuned on the composites at 640×640 with a batch size of 16. The class names were set to the Object IDs, so the model outputs IDs such as OBJ059 directly. On top of the composite generation, YOLOv8's built-in augmentations were used: HSV color shifts, translation, scaling and horizontal flips. Mosaic augmentation was reduced to 50%, because the images are already composites, and switched off for the final 10 epochs. Rotation was disabled to keep the boxes axis-aligned.

Training was allowed up to 80 epochs, with early stopping when validation mAP@0.5:0.95 stopped improving for 20 epochs. **It stopped after 71 epochs; the best model came from epoch 51**, and training took 62 minutes on a single NVIDIA T4 GPU. The validation curves (Figure 2) rise quickly in the first 15 epochs and then level off. The validation losses stay flat rather than rising, so there is no sign of overfitting.

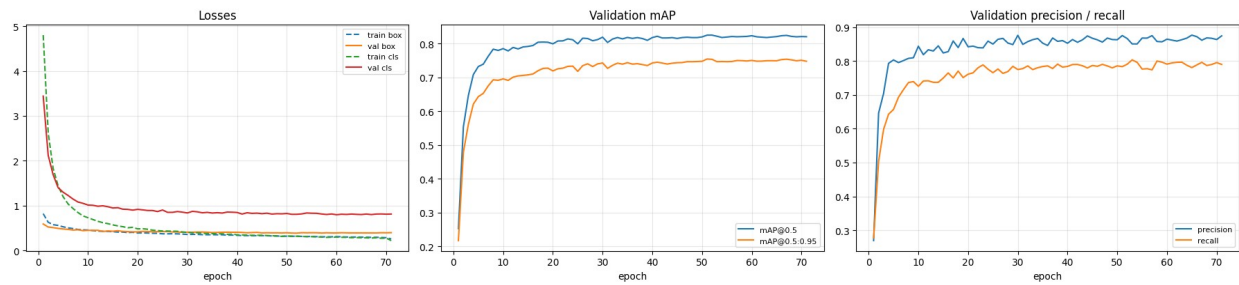


Figure 2. Training and validation losses, validation mAP, and validation precision and recall by epoch.

4. Results

All results are on the 500 held-out test images (2,309 objects). Besides the standard COCO-style mAP, an object-level evaluation was run at the deployment confidence threshold of 0.25. Under it, a detection counts as correct only if it has **the right Object ID and overlaps the true box with IoU ≥ 0.5** .

Metric	Value	What it means
mAP@0.5	0.821	Standard detection accuracy (box overlap $\geq 50\%$)
mAP@0.5:0.95	0.754	Averaged over stricter overlap thresholds
Precision / Recall / F1	0.846 / 0.811 / 0.828	Correct ID and location, at confidence ≥ 0.25
Localization recall (any ID)	0.830	Share of objects whose box was found, whatever the ID
ID accuracy of localized objects	0.977	Share of found objects given the correct Object ID
Object-count accuracy	0.630	Share of images where the number of detections equals the number of objects
Exact-image accuracy	0.342	Share of images with every object correct and nothing extra
Inference time	17.4 ms / image	Including pre- and post-processing (12.4 ms network)

Table 2. Test-set results.

Objects in image	2	3	4	5	6	7	8	9
Test images	98	74	83	79	81	25	37	23
Object recall	0.82	0.82	0.81	0.78	0.82	0.83	0.83	0.78
Exact-image accuracy	0.57	0.51	0.37	0.19	0.26	0.16	0.08	0.13

Table 3. Performance by number of objects per image.

Weakest Object IDs and confusions. The lowest test F1 scores were for OBJ004 Toy Car (0.48), OBJ003 NoteBook (0.55), OBJ010 Nike sneakers (0.56), OBJ034 Table Tennis Racket (0.57), OBJ070 Astronaut projector (0.57), OBJ065 Analog watch (0.60), OBJ060 Sleep Mask (0.63) and OBJ013 Mobile phone (0.64). Wrong IDs were rare. The most frequent confusion was OBJ070 Astronaut projector $\rightarrow$ OBJ017 Controller (5 times). Every other pair occurred at most twice, including OBJ059 Comb $\rightarrow$ OBJ061 Card wallet, which was also the most persistent confusion in Milestone 1.

Stress test. To check generalization beyond the training layouts, a new 4 $\times$ 3 composite with 12 objects (more than the training maximum of 9) was built from test-split photos. **The model identified all 12 objects with the correct Object IDs.**

5. Sample Detection Results

For any input image, the trained model returns each object's ID, name, confidence and box. In the notebook this is the function `detect_objects(image)`, and in the repository it is the script `predict.py`. Table 4 shows the output for test image `mo_test_00000`. Box coordinates are in pixels, with the origin at the top-left corner.

Object ID	Object name	Confidence	Box (x1, y1, x2, y2)	Center (x, y)
OBJ020	Airpod	0.915	(423, 119, 521, 175)	(472, 147)
OBJ011	mug	0.871	(424, 359, 471, 445)	(447, 402)
OBJ065	Analog watch	0.968	(11, 444, 217, 621)	(114, 533)

Table 4. Model output for test image `mo_test_00000` (Figure 4).

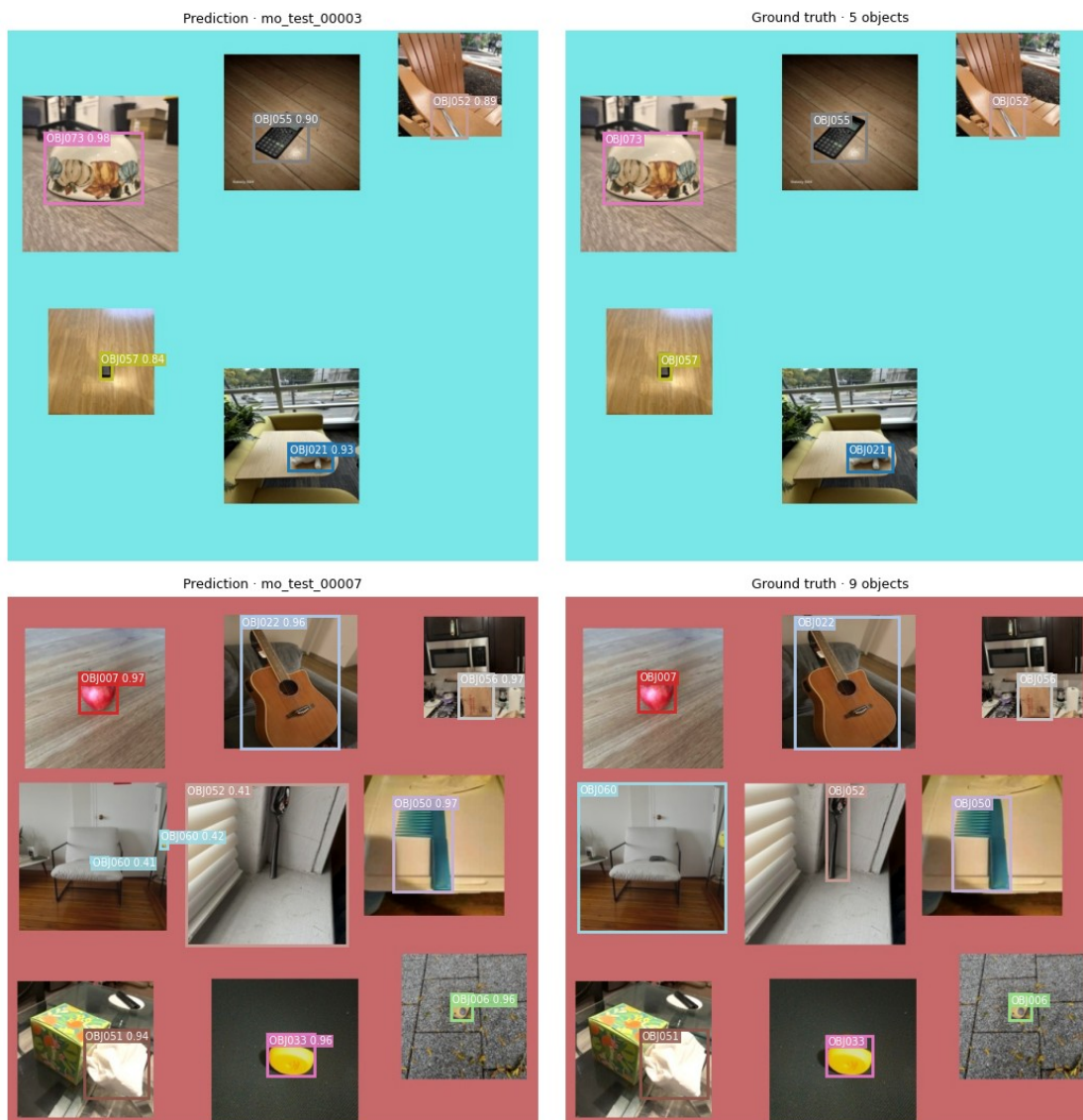


Figure 3. Predictions (left, with confidence) vs. ground truth (right). Top: all 5 objects correct, including the small OBJ057 Power Bank. Bottom: a 9-object image in which 7 objects are exactly right; the two misses are discussed in Section 6.

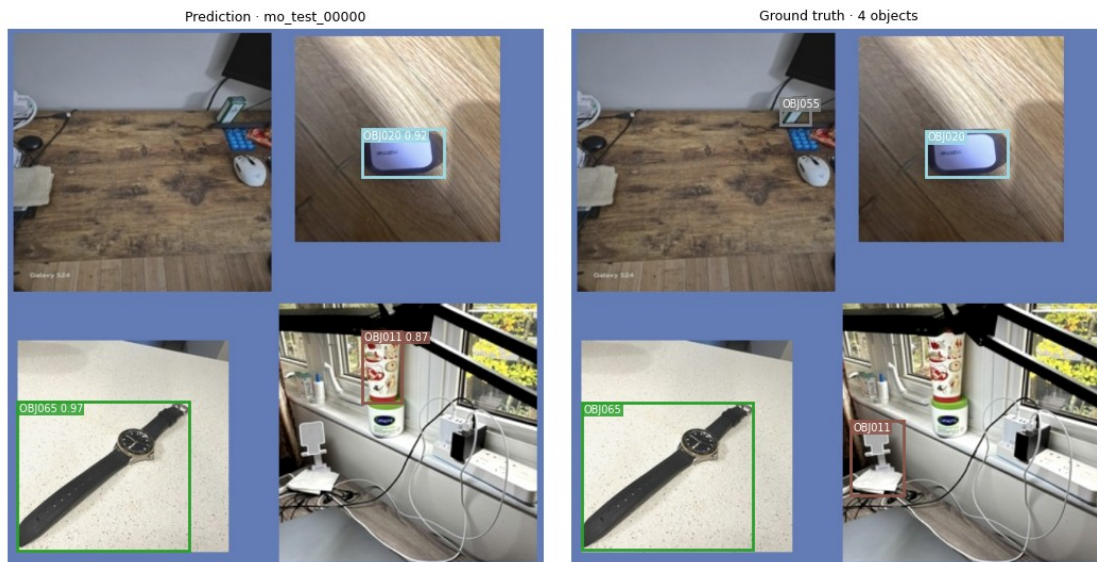


Figure 4. A harder example. OBJ020, OBJ065 and OBJ011 are found, and OBJ055 (a calculator occupying only a few pixels of its photo) is missed. For OBJ011, the model correctly boxes the mug, while the automatically generated ground-truth box marks a different item.

6. Discussion

The model names objects very reliably; nearly all errors are about the box. Of the objects it located, 97.7% received the correct Object ID, so recognizing which of the 73 objects is present is largely solved. The remaining errors are mostly objects that were missed or whose box did not overlap the reference box enough. The sample results point to three causes:

- **Inconsistent reference boxes.** For 13.6% of photos, OWLv2 could not find the object and the whole photo was used as the box. The model therefore learned two box styles: tight boxes for most objects, and whole-photo boxes for some. In Figure 3 (bottom), the model correctly identifies OBJ060 Sleep Mask and OBJ052 Spoon but draws a different box size from the reference, so both count as misses. The weakest Object IDs reflect this: OBJ004 Toy Car (45% fallback boxes) is the weakest class, and OBJ060 Sleep Mask (40%) is also in the bottom eight.
- **Errors in the reference boxes.** In Figure 4, the generated box for OBJ011 mug marks the wrong item, while the model boxes the actual mug. This is scored as both a miss and a false detection, so the reported metrics are, if anything, conservative.
- **Very small objects.** Some photos show their object small and far away. After scaling into a 3×3 grid, objects such as the calculator in Figure 4 cover only a handful of pixels.

More objects do not make the model worse at finding each one. Object recall stays between 0.78 and 0.83 from 2 to 9 objects per image (Table 3). Exact-image accuracy falls as objects are added only because each image has more chances for a single miss. With about 81% recall per object, the chance of getting all n objects right is roughly 0.81^n , which predicts the observed drop from 0.57 for 2 objects to 0.13 for 9.

Limitations. The composites are cleaner than real scenes: objects never overlap or occlude each other, each keeps its original photo background, and the reference boxes come from OWLv2 rather than manual annotation. Results come from a single training run. The most useful improvements would be correcting the boxes for the Object IDs with the most fallbacks (for example, OBJ043 Swimming Goggles at 82% and OBJ040 Clutcher at 56%), generating composites with overlapping objects and varied backgrounds, and trying a larger YOLOv8 model.

7. Code

All code is on GitHub: <https://github.com/imangoman/IE7615-Object-Recognition>. The repository contains the Milestone 2 notebook, which covers dataset generation, training, evaluation and detection; predict.py, which reports the Object IDs and boxes for any image from the command line; the final Object ID list; and the Milestone 1 notebook and report. The notebook runs end to end in Google Colab on a T4 GPU, and fixed random seeds make the dataset and split reproducible.